Dédicace

À la mémoire de mon très cher père, Demba Seck, parti bien trop tôt : que le Tout Puissant l'accueille en Sa sainte miséricorde, et que cette œuvre honore sa mémoire.

À ma mère bien-aimée, pour ses sacrifices immenses, son courage et cet amour qui n'a jamais faibli.

À ma tante Tacko Seck, qui m'a élevé avec l'amour d'une mère ; à son époux, mon père de cœur, et à leurs enfants, mes frères et sœurs.

À Sokhna Aicha Fatma Leye, ma cousine, dont la générosité et le dévouement ont illuminé mon parcours.

À toute ma famille et à mes camarades de promotion, pour leurs encouragements et leur soutien de chaque instant.

À mon guide religieux, Serigne Mountakha Bassirou Mbacké, que Dieu le préserve et l'assiste, pour l'université gratuite qu'il offre à la jeunesse, fidèle à l'héritage de Cheikh Ahmadou Bamba, fondateur du mouridisme et serviteur du Prophète Mouhamed (PSL).

« Ô Seigneur ! Fais de ma demeure un lieu propice à la quête du savoir, à la méditation et à la compréhension, et fais de Touba un foyer d'orientation, de droiture et d'enseignement. » (Cheikh Ahmadou Bamba, Matlabul Fawzayni)

Remerciements

Au terme de ce projet de fin de cycle, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont rendu ce travail possible.

Mes remerciements s'adressent en premier lieu à mon encadreur, Dr Abdoul Aziz CISS, pour la qualité de son encadrement, sa disponibilité et la pertinence de ses orientations tout au long de ce projet. Son exigence scientifique et sa bienveillance ont été pour moi un moteur constant.

Je remercie le corps enseignant et administratif de l'UFR des Métiers et Technologies (MET) de l'Université Cheikh Ahmadoul Khadim de Touba (UCAK), pour la formation reçue au sein du Département Informatique et Télécommunications (IT), option Administration et Sécurité des Réseaux.

Nos remerciements vont également à Monsieur le Directeur de l'UFR des Métiers et Technologies, Dr Mouhamadou Masseck Fall, ainsi qu'à tout le personnel administratif, pour le cadre de travail et les facilités accordés tout au long de notre formation.

Nous gardons une pensée émue pour le regretté Dr Pape Mory Gueye, alors chef du Département Informatique et Télécommunications, qui nous a quittés : que son âme repose en paix et que sa mémoire demeure une source d'inspiration. Nos remerciements s'adressent aussi à son successeur, Dr Malaw Ndiaye, pour son accueil, sa disponibilité et son accompagnement jusqu'à l'achèvement de ce travail.

Enfin, que tous ceux qui, de près ou de loin, ont contribué de quelque manière que ce soit à la réalisation de ce mémoire trouvent ici l'expression de ma sincère reconnaissance.

Résumé

Le présent mémoire traite de la conception et de la mise en place d'un système de gestion des vulnérabilités (VMS) pour l'Université Cheikh Ahmadoul Khadim (UCAK). Le travail s'appuie sur un laboratoire de réseau EVE-NG de quinze nœuds reproduisant une architecture de centre de données à double liaison de secours optique, avec deux bâtiments, des postes Windows et Windows 7, un serveur Ubuntu volontairement obsolète et une caméra IP connectée aux objets. Un scanner de vulnérabilités OpenVAS (pile Greenbone) est déployé au sein du réseau afin de réaliser des campagnes de découverte non authentifiées puis authentifiées. Les constats sont centralisés dans la plateforme DefectDojo, priorisés selon une approche fondée sur le risque (RBVM) croisant le score CVSS et une grille de criticité métier, puis transformés automatiquement en tickets Jira et en alertes Slack. La remédiation est assurée par des correctifs manuels ou par des manuels de tâches (playbooks) Ansible, et la boucle est refermée par des re-scans dont l'import clôt automatiquement les tickets. Un tableau de bord d'indicateurs (couverture, MTTR, tendance, âge moyen, taux d'automatisation) permet le pilotage en continu, et une campagne de non-régression ainsi qu'un test de continuité (bascule de la double liaison optique) valident la solution. Les résultats obtenus montrent une couverture de 100 % des actifs vitaux, un délai moyen de remédiation de 14 jours pour le cas critique de référence, un taux de remédiation supérieur à 1 et une absence totale de régression.

Mots clés : gestion des vulnérabilités ; OpenVAS / Greenbone ; DefectDojo ; CVSS ; RBVM ; Jira ; Slack ; Ansible ; EVE-NG ; MTTR ; KPI.

Abstract

This thesis presents the design and implementation of a Vulnerability Management System (VMS) for the Cheikh Ahmadoul Khadim University (UCAK). The work relies on a fifteen-node EVE-NG network laboratory reproducing a data-centre architecture with redundant fibre-optic uplinks, two buildings, Windows and Windows 7 workstations, a deliberately obsolete Ubuntu server and an IP camera. An OpenVAS scanner (Greenbone stack) is deployed inside the network to run unauthenticated then credentialed scans. Findings are centralised in DefectDojo, prioritised through a Risk-Based Vulnerability Management (RBVM) approach combining CVSS scores and a business-criticality grid, then automatically turned into Jira tickets and Slack alerts. Remediation is performed manually or through Ansible playbooks, and the loop is closed by re-scans whose import automatically closes the tickets. A KPI dashboard (coverage, MTTR, trend, average age, automation rate) supports continuous monitoring, while a non-regression campaign and a fibre-optic failover test validate the solution. Results show 100 % coverage of vital assets, a 14-day mean time to remediate for the reference critical case, a remediation ratio above 1 and zero regressions.

Keywords: vulnerability management; OpenVAS / Greenbone; DefectDojo; CVSS; RBVM; Jira; Slack; Ansible; EVE-NG; MTTR; KPI.

Abréviations et acronymes

API : Application Programming Interface (interface de programmation d'applications)

AWX : Ansible Works (interface open source d'Ansible Automation Platform)

CVE : Common Vulnerabilities and Exposures (référentiel des vulnérabilités connues)

CVSS : Common Vulnerability Scoring System (système de notation des vulnérabilités)

CSV : Comma-Separated Values (fichier de valeurs séparées par des virgules)

EPSS : Exploit Prediction Scoring System (score de probabilité d'exploitation)

EOL : End Of Life (fin de vie / fin de support d'un produit)

EVE-NG : Emulated Virtual Environment - Next Generation (simulateur de réseaux)

FO : Fibre Optique

GCE : Greenbone Community Edition (moteur de scan communautaire)

GSA : Greenbone Security Assistant (interface web de Greenbone)

GVM : Greenbone Vulnerability Management (pile de gestion des vulnérabilités)

IoT : Internet of Things (objets connectés)

IPSec : Internet Protocol Security (suite de protocoles de sécurité IP)

KPI : Key Performance Indicator (indicateur clé de performance)

MTTR : Mean Time To Remediate (délai moyen de remédiation)

NVT : Network Vulnerability Tests (tests de vulnérabilité réseau de Greenbone)

OS : Operating System (système d'exploitation)

PFC : Projet de Fin de Cycle

QoD : Quality of Detection (qualité de détection d'un constat)

RBVM : Risk-Based Vulnerability Management (gestion des vulnérabilités fondée sur le risque)

SMB : Server Message Block (protocole de partage Windows)

SSH : Secure Shell (protocole d'administration distante chiffrée)

SSL/TLS : Secure Sockets Layer / Transport Layer Security (protocoles de chiffrement)

SLA : Service Level Agreement (délai contractuel de traitement)

UCAK : Université Cheikh Ahmadoul Khadim de Touba

UFR MET : Unité de Formation et de Recherche des Métiers et Technologies

VLAN : Virtual Local Area Network (réseau local virtuel)

VMS : Vulnerability Management System (système de gestion des vulnérabilités)

VTY : Virtual TeletYpe (lignes d’administration distante des équipements IOS)

Sommaire

Dédicace

Remerciements

Résumé

Abstract

Abréviations et acronymes

Sommaire

Liste des figures

Liste des tableaux

Introduction générale

Chapitre 1 : Généralités sur la gestion des vulnérabilités

Chapitre 2 : Environnement du projet

Chapitre 3 : Détection, centralisation et priorisation des vulnérabilités (Objectifs 1 à 4)

Chapitre 4 : Le workflow de remédiation automatisé (Objectif 5)

Chapitre 5 : Le pilotage par les indicateurs (Objectif 6)

Chapitre 6 : Évaluation de l'efficacité et continuité de service (Objectif 7)

Conclusion générale et perspectives

Références bibliographiques

Webographie

Annexes

Table des matières

Liste des figures

Figure 1 : Architecture globale du réseau de référence ;

Figure 2 : Architecture interne d'un bâtiment ;

Figure 3 : Découverte réseau (L1), services telnet exposés sur les commutateurs ;

Figure 4 : Constats authentifiés du serveur cloud centralisés dans DefectDojo ;

Figure 5 : Constat critique 10,0 révélé par le scan authentifié sur WIN-10-1 ;

Figure 6 : Interrogation de l'API v2 de DefectDojo (curl authentifié) ;

Figure 7 : Configuration JIRA dans DefectDojo (correspondance des sévérités) ;

Figure 8 : Tickets créés automatiquement dans le projet VMS ;

Figure 9 : Notification d'import reçue dans le canal Slack #sec-alerts ;

Figure 10 : Sortie du playbook Ansible, changed=2 puis changed=0 (idempotence) ;

Figure 11 : Ticket VMS-80 clôturé automatiquement après le réimport de validation ;

Figure 12 : Planification quotidienne « Campus-W10 » dans la GSA ;

Figure 13 : Tableur KPI (indicateurs, cibles et historique des relevés) ;

Figure 14 : Tableau de bord DefectDojo (répartition des sévérités) ;

Figure 15 : Contrôle nmap des services d'administration (telnet fermé) ;

Figure 16 : Constats Windows clôturés en chaîne après réimport avec « Clôturer les anciennes » ;

Figure 17 : Le piège telnet (détection, ticket VMS-120 et clôture automatique) ;

Figure 18 : Bascule de la double liaison (tables de routage avant et après coupure) ;

Figure 19 : Tableau de bord final DefectDojo (20 constats clôturés en sept jours).

Figure 20 : Code QR du dépôt GitHub du projet.

Liste des tableaux

Tableau 1 : Les outils de la chaîne de gestion des vulnérabilités ;

Tableau 2 : Les actifs du laboratoire ;

Tableau 3 : Les vulnérabilités plantées par cible ;

Tableau 4 : Grille de criticité métier appliquée à l'université ;

Tableau 5 : Extraits de la matrice RBVM (Risque = CVSS × Criticité) ;

Tableau 6 : Les étapes du workflow et leurs outils ;

Tableau 7 : Le tableau de bord KPI et ses valeurs ;

Tableau 8 : Le bilan d’efficacité final ;

Tableau 9 : Principales difficultés rencontrées et solutions retenues.

Introduction générale

La numérisation accélérée des services universitaires (gestion administrative, plateformes de formation, réseaux de recherche, objets connectés) fait de l'infrastructure informatique un bien critique dont la compromission aurait des conséquences pédagogiques, administratives et réputationnelles majeures. Chaque serveur, chaque poste de travail et chaque objet connecté embarque des logiciels dont les failles de sécurité sont découvertes et publiées quotidiennement dans des référentiels publics : gérer ces vulnérabilités n'est plus une option, c'est la condition première de la sécurité du système d'information.

La gestion des vulnérabilités désigne précisément ce processus continu qui consiste à identifier les failles présentes sur les actifs du réseau, à les évaluer selon leur gravité technique et leur importance métier, à les corriger de manière maîtrisée, puis à vérifier que les corrections tiennent dans le temps. Loin d'être un simple passage d'outils de scan, il s'agit d'un véritable cycle de vie qui englobe la détection, la centralisation, la priorisation, la remédiation, le pilotage par les indicateurs et la validation de la non-régression.

C'est dans ce cadre que s'inscrit le présent projet de fin de cycle, intitulé « Conception et mise en place d'un système de gestion des vulnérabilités pour l'Université Cheikh Ahmadoul Khadim (UCAK) ». Le travail s'appuie sur un laboratoire de virtualisation de réseau EVE-NG reproduisant, à l'échelle, une architecture de centre de données à double liaison opérateur en fibre optique : un routeur de cœur, deux bâtiments représentatifs, un nuage IaaS exposable, ainsi que des postes de travail, un serveur volontairement obsolète et une caméra IP. Cette maquette permet de conduire l'intégralité de la démarche (de la première découverte réseau jusqu'à la preuve de non-régression) dans des conditions contrôlées et reproductibles.

Le cahier des charges se décline en sept objectifs opérationnels : (1) maîtriser le cycle de vie d'une vulnérabilité ; (2) déployer un scanner de vulnérabilités dans le laboratoire et réaliser les premiers scans ; (3) mettre en place un système de scoring et de priorisation fondé sur le CVSS et le contexte métier (approche RBVM) ; (4) construire un workflow de remédiation automatisé articulant centralisation, ticketing, notification et correctifs ; (5) doter la plateforme d'un tableau de bord d'indicateurs ; (6) évaluer l'efficacité de la solution par une campagne de non-régression ; (7) vérifier la continuité de service de la plateforme lors de la bascule de la double liaison.

La méthodologie retenue est expérimentale et itérative : chaque objectif donne lieu à un livrable documenté, adossé à des preuves horodatées (rapports de scan, captures d'écran, sorties de commandes, tableaux d'indicateurs). Le mémoire suit cette progression : le chapitre 1 pose les généralités et le cycle de vie de la gestion des vulnérabilités ; le chapitre 2 décrit l'environnement matériel et les cibles du laboratoire ; le chapitre 3 présente la détection, la centralisation et la priorisation ; le chapitre 4 développe le workflow de remédiation automatisé ; le chapitre 5 expose le pilotage par les indicateurs ; le chapitre 6 évalue l'efficacité de la solution et sa continuité de service ; la conclusion dresse le bilan et ouvre les perspectives.

Chapitre 1 : Généralités sur la gestion des vulnérabilités

Section 1 : Définition de quelques concepts clés

Une vulnérabilité est une faiblesse d'un système d'information (défaut de conception, erreur de configuration, logiciel obsolète ou mauvaise pratique) susceptible d'être exploitée pour porter atteinte à la confidentialité, à l'intégrité ou à la disponibilité des données. Lorsqu'elle est publiée, elle reçoit un identifiant CVE (Common Vulnerabilities and Exposures) dans le référentiel tenu par MITRE, ce qui permet de la suivre de manière univoque d'un outil à l'autre.

Sa gravité technique est mesurée par le CVSS (Common Vulnerability Scoring System), maintenu par le consortium FIRST : le score, compris entre 0 et 10, synthétise la facilité d'exploitation et l'impact potentiel (les fourchettes usuelles étant Low de 0,1 à 3,9, Medium de 4,0 à 6,9, High de 7,0 à 8,9 et Critical de 9,0 à 10). Le score CVSS est toutefois générique : une faille de 9,8 sur un poste de test isolé ne présente pas le même risque que la même faille sur le serveur de base de données de production. C'est pourquoi l'approche moderne, dite RBVM (Risk-Based Vulnerability Management), croise la sévérité technique avec la criticité métier de l'actif concerné pour calculer un niveau de risque et prioriser les actions. En complément, l'EPSS (Exploit Prediction Scoring System) estime la probabilité qu'une vulnérabilité soit effectivement exploitée dans la nature, et la Quality of Detection (QoD) renseigne sur la fiabilité avec laquelle le scanner a détecté le constat.

Enfin, la remédiation s'inscrit dans des délais cibles formalisés par des SLA (Service Level Agreement), par exemple trente jours pour un constat critique, dont le respect se mesure à l'aide du MTTR (Mean Time To Remediate), délai moyen écoulé entre la première détection d'une faille et sa clôture après correction vérifiée.

Section 2 : Standards et référentiels

La démarche adoptée s'appuie sur les référentiels suivants : le NIST SP 800-40 Rev. 3, qui structure le processus d'entreprise de gestion des correctifs (inventaire, priorisation, déploiement, vérification) ; la norme ISO/IEC 27001:2022, dont l'annexe A traite de la maîtrise « gestion des vulnérabilités technologiques » ; la spécification CVSS v3.1 du FIRST pour la notation ; les référentiels CVE (MITRE) et NVD (NIST) pour l'identification et l'enrichissement des failles ; et le système EPSS du FIRST pour l'anticipation de l'exploitation. Ces référentiels fournissent le vocabulaire commun et les garde-fous méthodologiques appliqués tout au long du projet.

Section 3 : Les outils de la chaîne

La chaîne de gestion des vulnérabilités mise en place articule six outils open source ou gratuits, choisis pour leur maturité, leur interopérabilité native et leur adéquation au contexte d'une université. Le tableau 1 en résume les rôles.

Tableau 1 : Les outils de la chaîne de gestion des vulnérabilités

Outil	Rôle dans la chaîne
OpenVAS / Greenbone (GCE)	Scanner de vulnérabilités : découverte réseau, scans authentifiés, plus de 180 000 tests NVT
Nmap	Découverte d'actifs, inventaire, contrôles ciblés de ports et de services
DefectDojo (Community)	Centralisation des scans, déduplication, moteur de notification, API v2, SLA, métriques
Jira Cloud	Ticketing : création automatique des tickets de remédiation et suivi des statuts
Slack	Notification instantanée des événements (import de scan, clôture de tickets)
Ansible	Automatisation des correctifs (playbooks idempotents) sur les cibles Linux / objets connectés

Le choix de DefectDojo Community Edition s'est confirmé au fil du projet : l'édition gratuite intègre plus de cinq cents parsers de rapports (dont le format CSV de Greenbone), l'intégration Jira native, les notifications Slack, une API v2 complète, le suivi des SLA et la déduplication, soit l'intégralité du workflow visé, l'offre commerciale n'ajoutant que des connecteurs premium et du support.

Section 4 : Le cycle de vie de la vulnérabilité

Le cycle de vie mis en œuvre dans ce projet comporte six phases, qui structurent les chapitres suivants : (1) l'inventaire et la découverte des actifs ; (2) la détection, par des scans non authentifiés puis authentifiés ; (3) la centralisation et la déduplication des constats dans une plateforme unique ; (4) la priorisation fondée sur le risque (CVSS × criticité métier) ; (5) la remédiation, manuelle ou automatisée, tracée par des tickets et des alertes ; (6) la vérification, par re-scan et clôture automatique, complétée par le pilotage d'indicateurs et la non-régression. Ce cycle n'est pas linéaire mais permanent : la phase 6 réinjecte la phase 2 à la fréquence des planifications.

Chapitre 2 : Environnement du projet

Section 1 : L'architecture réseau de référence

L'infrastructure de référence est organisée autour d'un centre de données interconnecté à l'opérateur par deux liaisons spécialisées en fibre optique : une liaison principale raccordée au central MADIYANA et une liaison de secours raccordée au central NDAM-MBACKÉ. Cette redondance assure la continuité de la connectivité Internet : la perte de la liaison principale doit déclencher le basculement automatique vers la liaison de secours, sans interruption des services ni des opérations de surveillance. Le centre de données abrite le routeur de cœur, le commutateur principal et les équipements de distribution optique (ODF) ; de ce point central partent les fibres desservant les bâtiments (SMS, MET, ETIS, LLA, Touba Ca Kanam, Majalis, Restaurant), chacun équipé de son ODF et de son commutateur d'accès. L'ensemble adopte une topologie en étoile dont le centre de données est le nœud central, ce qui facilite l'administration, la maintenance et l'extension.

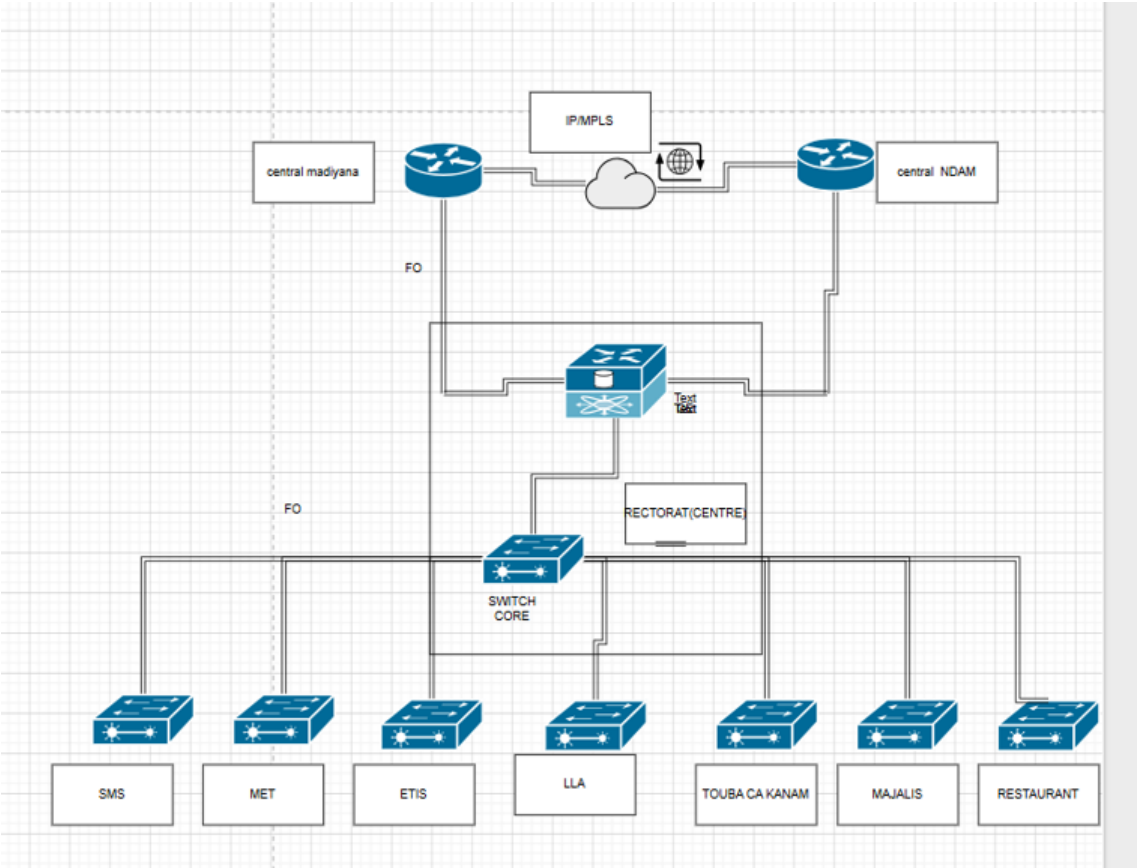


Figure 1 : Architecture globale du réseau de référence

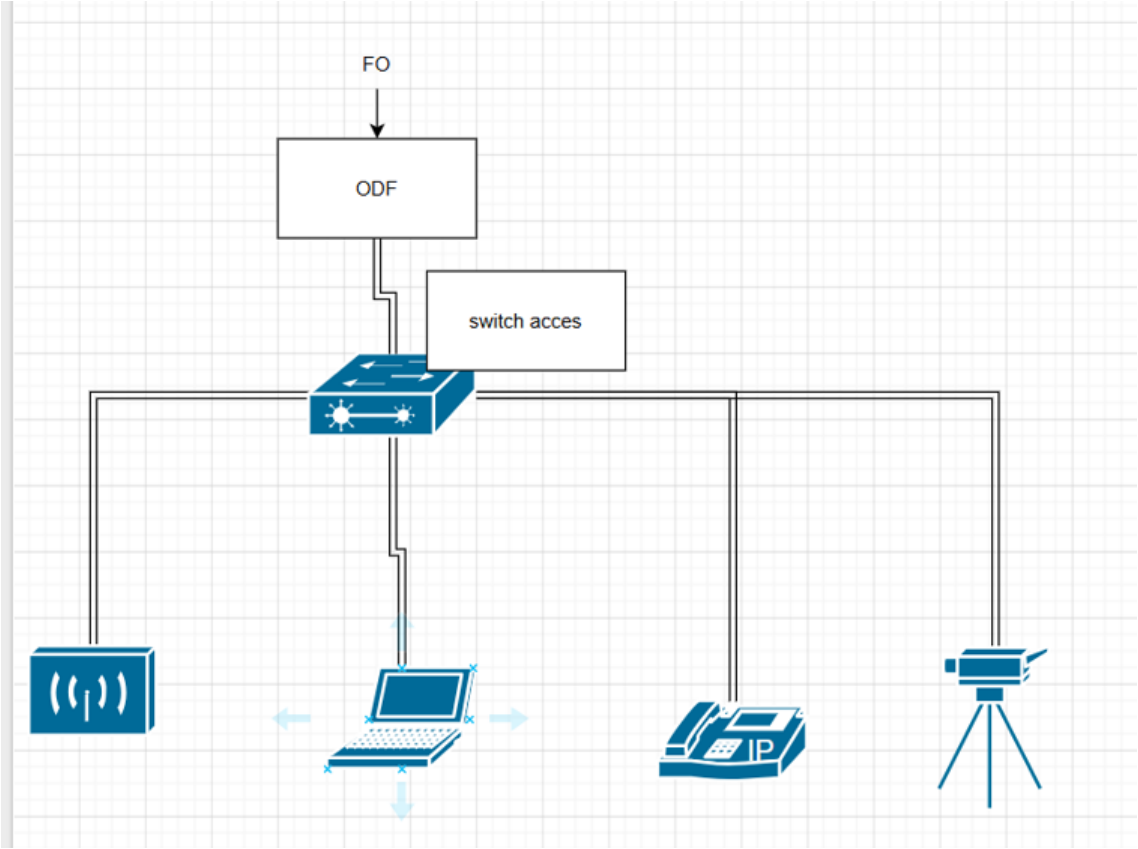


Figure 2 : Architecture interne d'un bâtiment

C'est cette architecture qui est reproduite au laboratoire : le routeur de cœur, les deux sorties opérateur (route flottante de secours), deux bâtiments représentatifs câblés sur des VLAN distincts, un commutateur d'administration et un nuage IaaS susceptible d'exposer des services vers l'extérieur. Cette reproduction fidèle est essentielle : les constats et les remédiations démontrés au laboratoire sont directement transposables au réseau de production.

Section 2 : Le laboratoire EVE-NG

Le laboratoire repose sur un hôte physique de 16 Go de mémoire exécutant EVE-NG (8 Go alloués au simulateur), et compte quinze nœuds plus deux nuages. Le scanner de vulnérabilités est une machine virtuelle Kali Linux (GBVA) dotée de 4 Go de mémoire, équipée de la pile Greenbone (OpenVAS) installée par les dépôts de la distribution ; disposant de deux interfaces, elle est le seul nœud à la fois routé vers toutes les cibles du laboratoire et connecté au réseau local de l'hôte, condition nécessaire pour télécharger les mises à jour des tests NVT et pour héberger les automatisations (Ansible, gvm-tools). L'interface web du scanner (GSA) est exposée sur le port 9392.

Les actifs surveillés, résumés au tableau 2, couvrent volontairement toute la diversité d'un parc universitaire : postes de travail récents et anciens, serveur virtualisé, objet connecté et équipements d'interconnexion.

Tableau 2 : Les actifs du laboratoire

Actif	Adresse	Rôle / OS	Observations
SRV-CLOUD	10.9.1.10	Serveur Ubuntu 20.04 (EOL)	Cloud IaaS (accessible depuis l'extérieur) ; OS gelé volontairement (scénario d'étude)
WIN-TC1	172.16.30.50	Windows 7 (poste bâtiment 1)	Mot de passe administrateur faible planté au départ
WIN-TC2	172.16.31.10	Windows 7 (poste bâtiment 2)	Compte utilisateur par défaut
WIN-ADMIN-1	172.16.30.10	Windows 10 (poste d'administration)	Poste de référence : aucune faiblesse plantée (base de comparaison)
WIN-10-1	172.16.31.11	Windows 10 (poste bâtiment 2)	Mot de passe administrateur vide planté au départ
CAMERA	172.16.30.11	Caméra IP (Linux embarqué)	Serveur web d'administration et SSH
PHONE	172.16.30.60	Téléphone IP	Découverte (pas d'authentification possible)
SW-MET / SW-TOR	SVI VLAN 30/31	Commutateurs IOSv	Administration distante (telnet fermé après remédiation)
GBVA	172.16.10.10	Scanner Kali + Greenbone	2 interfaces ; Ansible et gvm-tools

Section 3 : Préparation des cibles et vulnérabilités plantées

Pour que la détection soit démontrable, les cibles ont été volontairement affaiblies avant les premiers scans, de manière documentée et réversible. Le tableau 3 récapitule les principales faiblesses plantées, qui constituent la « vérité terrain » des campagnes de détection.

Tableau 3 : Les vulnérabilités plantées par cible

Cible	Vulnérabilités plantées
WIN-TC1 / WIN-TC2 (Windows 7)	Partage SMBv1 actif (exposition MS17-010), protocole SMB exposé à une exécution de code à distance, RDP ouvert (BlueKeep), mots de passe faibles, système en fin de support
WIN-ADMIN-1 / WIN-10-1 (Windows 10)	Mot de passe administrateur vide ou faible, protocoles TLS anciens acceptés, partages administratifs exposés
SRV-CLOUD (Ubuntu 20.04 EOL)	sudo vulnérable (CVE-2021-3156, Baron Samedit), noyau non corrigé (mitigations manquantes), clé d'hôte et comptes de service faibles ; le noyau reste volontairement non corrigé (OS gelé)
CAMERA (IoT)	Page d'administration par défaut, mot de passe admin/1234, SSH accessible
GSA (scanner)	Compte administrateur admin/admin planté pour le constat V-07, puis renforcé
Commutateurs IOS	Telnet d'administration ouvert au départ (constat L1), fermé par remédiation puis replanté volontairement au chapitre 6 (piège)

Chapitre 3 : Détection, centralisation et priorisation (Objectifs 1 à 4)

Section 1 : La découverte réseau (L1)

La première campagne est un scan non authentifié de découverte, mené le 5 octobre 2026 depuis GBVA sur les deux bâtiments. Elle établit l'inventaire des actifs vivants et révèle la surface exposée : services d'administration distante ouvertes, protocoles non chiffrés, partages Windows visibles. La figure 3 montre un exemple caractéristique : le service telnet exposé sur les interfaces de commutation des bâtiments, protocole obsolète qui transmet les identifiants en clair. Cette campagne compte trois constats critiques et un constat haut ; elle sert de référence pour mesurer l'apport de l'authentification.

Vulnerability		Severity	QoD	Host		Location	EPSS	
				IP	Name		Score	Percentile
Telnet Unencrypted Cleartext Login		4.8 (Medium)	70 %	172.16.30.1	172.16.30.1	23/tcp	N/A	N/A
Telnet Unencrypted Cleartext Login		4.8 (Medium)	70 %	172.16.255.1	172.16.255.1	23/tcp	N/A	N/A
Telnet Unencrypted Cleartext Login		4.8 (Medium)	70 %	172.16.31.2	172.16.31.2	23/tcp	N/A	N/A

Figure 3 : Découverte réseau (L1), services telnet exposés sur les commutateurs

Section 2 : Les scans authentifiés (L2)

La seconde campagne, datée du 13 octobre 2026, reprend les mêmes cibles avec des identifiants d'audit : comptes SMB pour les postes Windows, SSH pour le serveur et la caméra, SNMP pour le réseau. Le gain est spectaculaire : le scanner se connecte aux systèmes et évalue leurs configurations internes, révélant neuf constats critiques et trois constats hauts, soit presque le triple de la découverte. Les failles locales (sudo vulnérable sur le serveur, SMBv1 et MS17-010 sur les postes Windows 7, comptes par défaut, mot de passe administrateur vide) ne sont visibles qu'ainsi. La figure 4 montre les treize constats centralisés du serveur cloud, et la figure 5 un constat critique de niveau 10,0 (connexions SMB avec identifiants par défaut) révélé par l'authentification sur WIN-10-1.

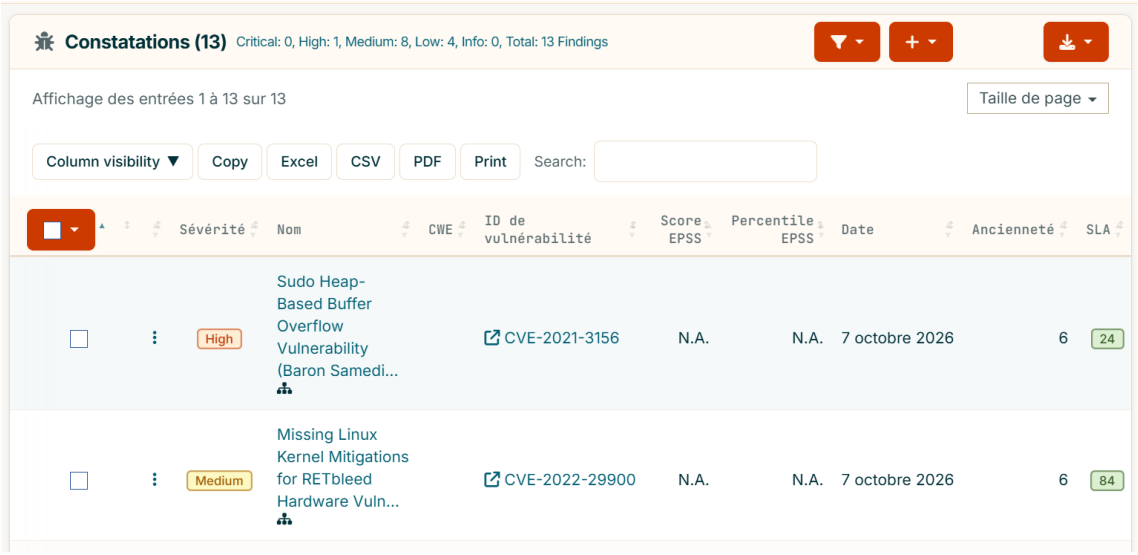


Figure 4 : Constats authentifiés du serveur cloud centralisés dans DefectDojo

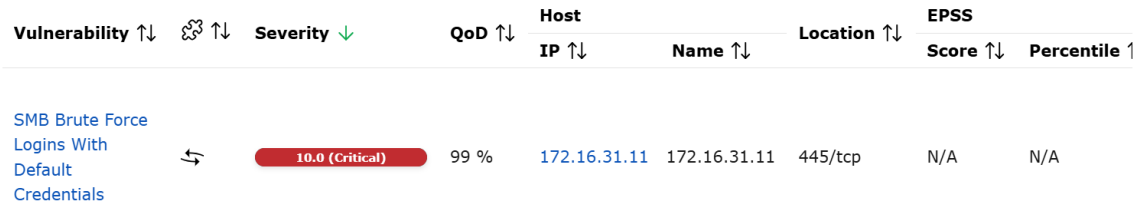


Figure 5 : Constat critique 10,0 révélé par le scan authentifié sur WIN-10-1

Cette comparaison L1/L2 fonde une règle de méthode du projet : la surface vue par un scan non authentifié n'est que la partie émergée de l'exposition réelle ; les campagnes de référence doivent être authentifiées, ce qui impose en contrepartie une discipline de mise à jour des identifiants d'audit à chaque changement de mot de passe ; les identifiants désynchronisés ont d'ailleurs été, au fil du projet, une source d'échecs documentée.

Section 3 : La centralisation dans DefectDojo

Les rapports de scan sont exportés de la GSA au format CSV (format Greenbone) puis importés dans DefectDojo, installé en Docker sur l'hôte du laboratoire (édition Community 3.4.0, interface sur le port 8080). Chaque campagne correspond à un engagement nommé de façon explicite (« AVANT remédiation : »), rattaché au produit concerné, avec l'environnement « Lab » et l'option « Verified = Force to True » qui marque les constats comme vérifiés (condition nécessaire, comme on le verra, au déclenchement du ticketing). La déduplication de la plateforme fait le reste : un constat réimporté ne crée pas de doublon mais met à jour la fiche existante en conservant sa première date de détection, propriété essentielle pour mesurer honnêtement l'âge des vulnérabilités. Une API v2 complète permet enfin d'interroger la plateforme par programme (figure 6).

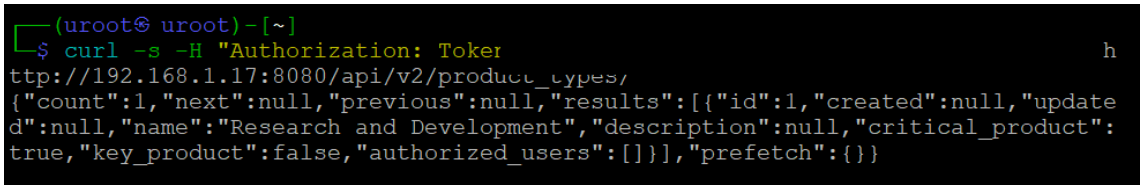


Figure 6 : Interrogation de l'API v2 de DefectDojo (curl authentifié)

Section 4 : Le scoring RBVM

Prioriser, c'est décider quoi corriger d'abord avec des ressources limitées. La grille de criticité métier retenue (tableau 4) note chaque actif de 1 à 5 selon la confidentialité, l'intégrité et la disponibilité des données qu'il porte, une fois pour toutes à l'inventaire. Le risque est alors calculé par la formule : Risque = Sévérité CVSS × Criticité métier. Trois priorités d'action en découlent : P1 (risque ≥ 25, traitement immédiat), P2 (10 ≤ risque < 25, traitement planifié) et P3 (risque < 10, traitement différé ou acceptation).

Tableau 4 : Grille de criticité métier appliquée à l'université

Criticité	Type d'actif à l'université	Exemples
5 : Vital	Serveurs de données, services transversaux	Serveur de base de données, serveur de fichiers central
4 : Majeur	Serveurs applicatifs, équipements d'interconnexion	Serveur applicatif, routeur de cœur, commutateurs
3 : Important	Postes d'administration, serveurs de déploiement	Poste administrateur, caméra d'infrastructure
2 : Courant	Postes de travail utilisateurs	Postes des salles, téléphones IP
1 : Mineur	Équipements isolés, bancs d'essai	Poste de test, objet connecté non sensible

La matrice consolidée (livrable 3) retient ainsi 39 constats de priorité 1 et 50 constats de priorité 2, répartis par actif et par scénario. Le tableau 5 en présente des extraits caractéristiques : on y lit l'effet de la criticité : un même CVSS ne produit pas la même priorité selon l'actif porteur.

Tableau 5 : Extraits de la matrice RBVM (Risque = CVSS × Criticité)

Constat (extrait)	CVSS	Criticité métier	Risque	Priorité
Exécution de code à distance SMB (Windows 7, bâtiment 1)	10,0	2	20	P2
MS17-010 : raccourci d'exécution SMB (Windows 7)	8,8	2	17,6	P2
BlueKeep : RDP (Windows 7)	9,8	2	19,6	P2
sudo Baron Samedit (serveur cloud)	7,8	4	31,2	P1
Comptes par défaut / mot de passe vide (Windows 10)	10,0	2-3	20-30	P1-P2
Telnet en clair (commutateurs)	4,8	4	19,2	P2

Chapitre 4 : Le workflow de remédiation automatisé (Objectif 5)

Section 1 : Architecture du workflow

Le workflow relie les outils entre eux pour que chaque détection au-delà du seuil de risque déclenche, sans intervention manuelle, la création d'un ticket, l'alerte des administrateurs, le déploiement du correctif puis la clôture vérifiée. Le tableau 6 résume les étapes et leurs outils ; les sections suivantes détaillent chacune d'elles, en distinguant soigneusement ce que la plateforme fait nativement de ce qui a exigé une investigation, car plusieurs documentations d'outils se sont révélées obsolètes, et une partie du travail a consisté à vérifier le comportement réel du code.

Tableau 6 : Les étapes du workflow et leurs outils

Étape	Outil	Comportement mis en œuvre
Centralisation et tri	DefectDojo	Import des CSV Greenbone, déduplication, SLA, seuil de priorité
Ticketing	Jira Cloud (projet VMS)	Création automatique des tickets pour les constats Actifs et Vérifiés au-delà du seuil Medium
Notification	Slack (canal #sec-alerts)	Alerte instantanée à chaque import de scan, via le bot natif de DefectDojo
Correctifs	Ansible (depuis GBVA)	Playbooks idempotents de remédiation (caméra) ; correctifs manuels documentés ailleurs
Clôture	DefectDojo → Jira	Réimport avec « Clôturer les anciennes » : les constats corrigés ferment leurs tickets

Section 2 : Le ticketing automatique avec Jira

L'intégration Jira de DefectDojo repose sur trois réglages : une instance Jira (URL du site cloud, type de ticket « Bug », projet de réception) ; une correspondance des sévérités (figure 7) qui traduit les niveaux de DefectDojo vers les priorités Jira ; et un seuil global (Medium dans notre cas) en dessous duquel aucun ticket n'est créé. Le déclenchement dépend en outre d'une subtilité essentielle constatée au laboratoire : seuls les constats à la fois Actifs et Vérifiés sont poussés lorsque l'option « Push All Issues » est activée sur le produit ; d'où la règle d'import « Verified = Force to True » retenue au chapitre précédent. Une fois ces réglages en place, chaque constat au-dessus du seuil a son ticket : le projet VMS a ainsi reçu automatiquement les tickets VMS-80 à VMS-88 pour le serveur cloud, VMS-89 à VMS-100 pour les postes Windows 7, puis VMS-101 à VMS-115 pour les postes Windows 10 (figure 8), chacun portant le lien vers le constat détaillé.

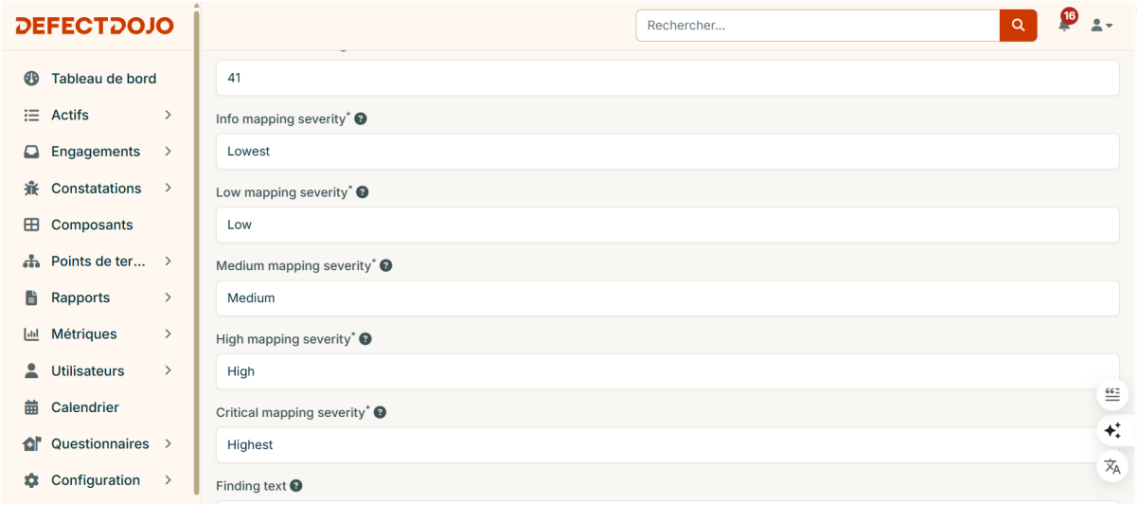


Figure 7 : Configuration JIRA dans DefectDojo (correspondance des sévérités)

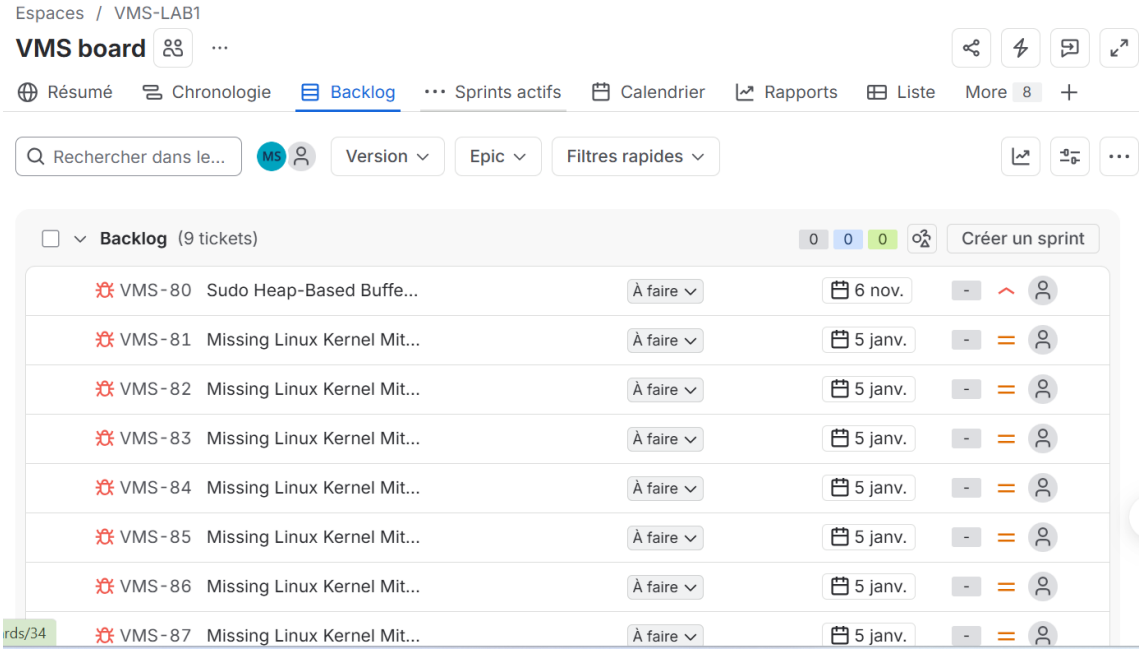


Figure 8 : Tickets créés automatiquement dans le projet VMS

Deux écueils d'intégration méritent d'être consignés car ils coûtèrent plusieurs itérations : les jetons d'API doivent être des jetons classiques d'utilisateur créés sur le portail de compte Atlassian (les clés d'administration de l'organisation sont refusées par l'API du site, avec des erreurs 401 trompeuses) ; et l'endpoint de métadonnées de Jira ne couvre pas les projets « gérés par l'équipe » (team-managed), ce qui provoque une erreur « Misconfigured default issue type » à la liaison ; la solution étant de recréer le projet « géré par l'entreprise » (company-managed), ce qui fut fait avec la clé VMS.

Section 3 : Les notifications Slack natives

La notification instantanée vise à alerter l'équipe dès l'import d'un scan. La documentation consultée orientait vers un webhook entrant paramétré par variable d'environnement ; or la configuration restait silencieuse. L'inspection du code source de DefectDojo 3.4 a tranché : cette

version envoie ses notifications via l'API Web de Slack, avec un appel codé en dur vers l'endpoint slack.com/api/chat.postMessage et des identifiants (jeton de bot, canal) stockés en base : la variable d'environnement n'est jamais lue. La voie native a donc été mise en œuvre : création d'une application Slack « defectdojo » dans le workspace du projet, attribution du droit chat:write, installation puis récupération du jeton de bot (préfixe xoxb), invitation du bot dans le canal #sec-alerts, et saisie du jeton, du canal et du nom d'affichage dans les réglages système de DefectDojo ; la figure 9 montre la notification d'import reçue dans le canal.

Un second réglage s'est révélé décisif : la matrice des notifications propose deux portées. La portée « personnelle » envoie des messages privés à l'utilisateur connecté ; c'est la portée « système » qui alimente un canal d'équipe. Après bascule de la règle « Scan added » vers Slack en portée système et sauvegarde, chaque import a produit le message attendu dans #sec-alerts : titre de l'engagement, type de parser, lien vers le test ; le lien utilisant l'adresse locale du serveur, valable depuis l'hôte. Une validation préalable au curl (appel direct à chat.postMessage avec le jeton) permet de dissocier proprement les causes : connectivité, invitation du bot, jeton et règle de notification. À titre de démonstration complémentaire, la même chaîne a été éprouvée vers un Mattermost local par webhook entrant, confirmant l'universalité du concept de canal d'alerte.

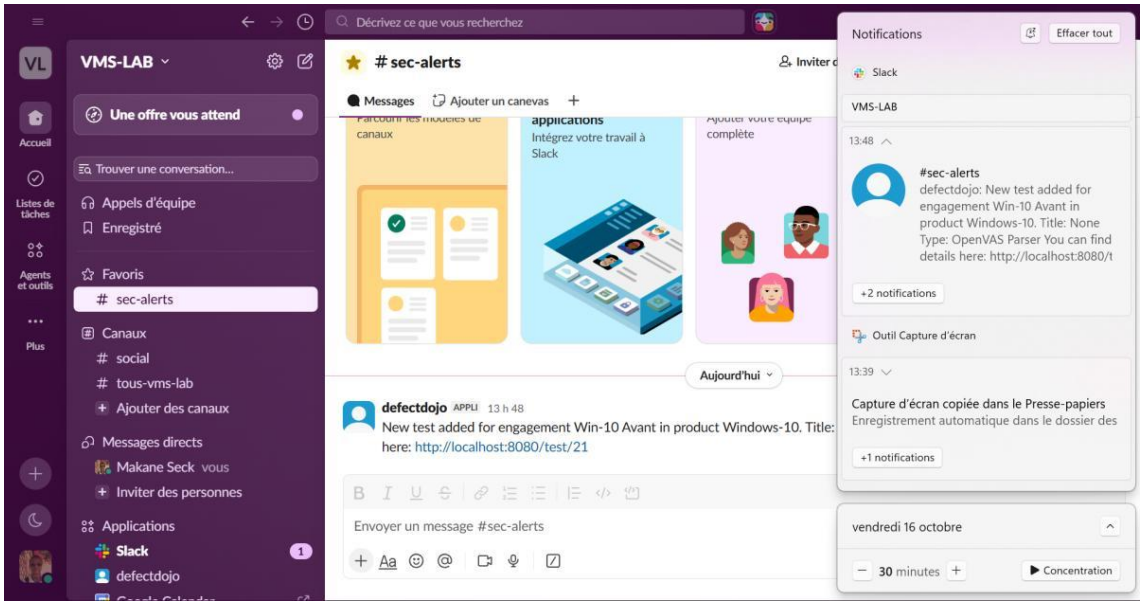


Figure 9 : Notification d'import reçue dans le canal Slack #sec-alerts

Section 4 : La remédiation par Ansible

La remédiation automatisée cible la caméra IP, objet connecté représentatif des périphériques difficiles : système Linux embarqué, Python trop ancien pour exécuter les modules Ansible modernes. Le playbook retenu contourne cette limite en s'appuyant exclusivement sur des tâches « raw » (commandes brutes sans interpréteur distant), avec élévation par sudo et des fichiers marqueurs pour l'idempotence : première exécution, le mot de passe fort est appliqué et la page d'administration par défaut est remplacée par une page durcie (deux tâches « changed ») ; seconde exécution, tout est déjà conforme et plus rien ne change (zéro tâche modifiée). Cette

démonstration d'idempotence (propreté fondamentale d'Ansible, qui garantit que rejouer un correctif ne produit pas d'effet de bord) est visible sur la sortie du playbook (figure 10). Le hash du mot de passe est calculé avec un sel fixe, condition nécessaire de l'idempotence du module utilisateur.

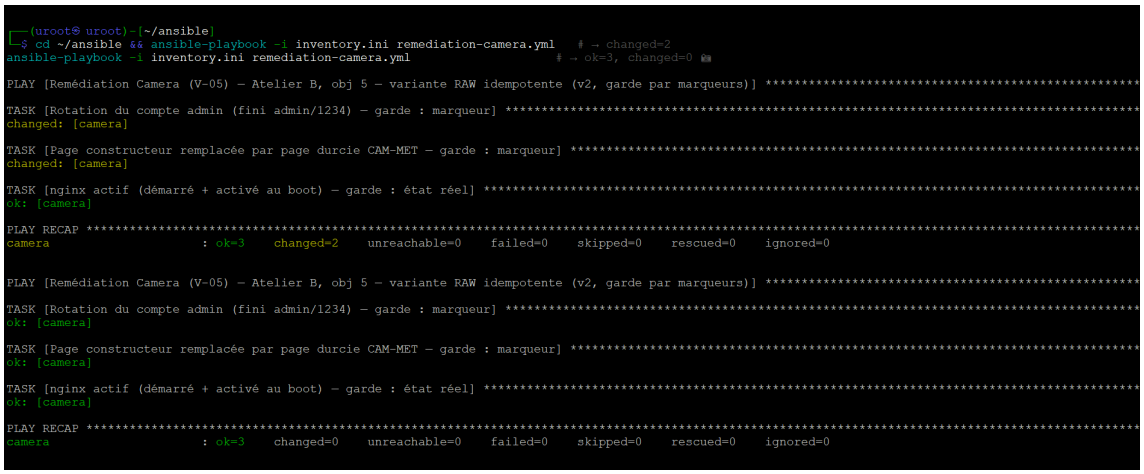


Figure 10 : Sortie du playbook Ansible, changed=2 puis changed=0 (idempotence)

Section 5 : La boucle de clôture automatique

La boucle de rétroaction referme le cycle : une fois le correctif appliqué, un re-scan est importé dans le même engagement avec l'option « Clôturer les anciennes ». Les constats absents du nouveau rapport sont clos automatiquement dans DefectDojo, et la synchronisation Jira fait suivre : le ticket passe tout seul à « Done ». Ce mécanisme a été prouvé deux fois : d'abord sur le serveur cloud, où le réimport du 16 octobre, après correction manuelle de sudo, a fait passer le ticket VMS-80 à Done sans aucune intervention dans Jira (figure 11) ; puis à l'échelle, lors de la campagne de clôture du 22 octobre qui a refermé vingt constats d'un coup, toutes les protections étant réimportées à l'identique pour ne pas altérer les fiches restées valides.

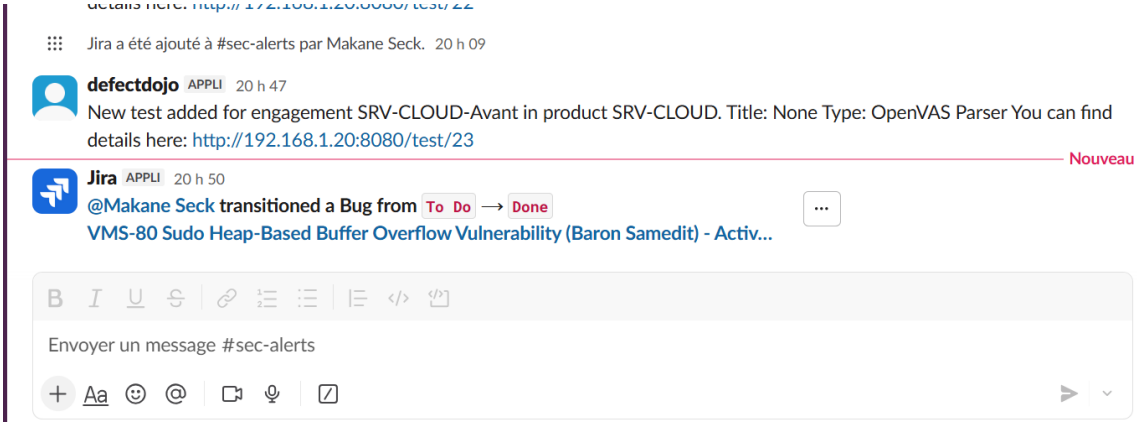


Figure 11 : Ticket VMS-80 clôturé automatiquement après le réimport de validation

Chapitre 5 : Le pilotage par les indicateurs (Objectif 6)

Section 1 : Les scans planifiés

La gestion des vulnérabilités est un processus continu : les scans ne peuvent donc pas rester des opérations manuelles. Cinq planifications ont été créées dans la GSA, selon une architecture dictée par deux contraintes réelles : la GSA n'accepte qu'un seul identifiant SMB/SSH par tâche, et la mémoire du scanner (4 Go) impose de ne pas cumuler les pics de charge : quatre tâches quotidiennes décalées par identifiant pour le campus (WIN-10 à 06:00, TC1 à 06:45, TC2 à 07:30, caméra à 08:15), et un audit complet authentifié du serveur cloud le dimanche à 02:00. Les identifiants SNMP de découverte s'ajoutent à chaque tâche dans leur champ dédié. La figure 12 montre le détail d'une planification.

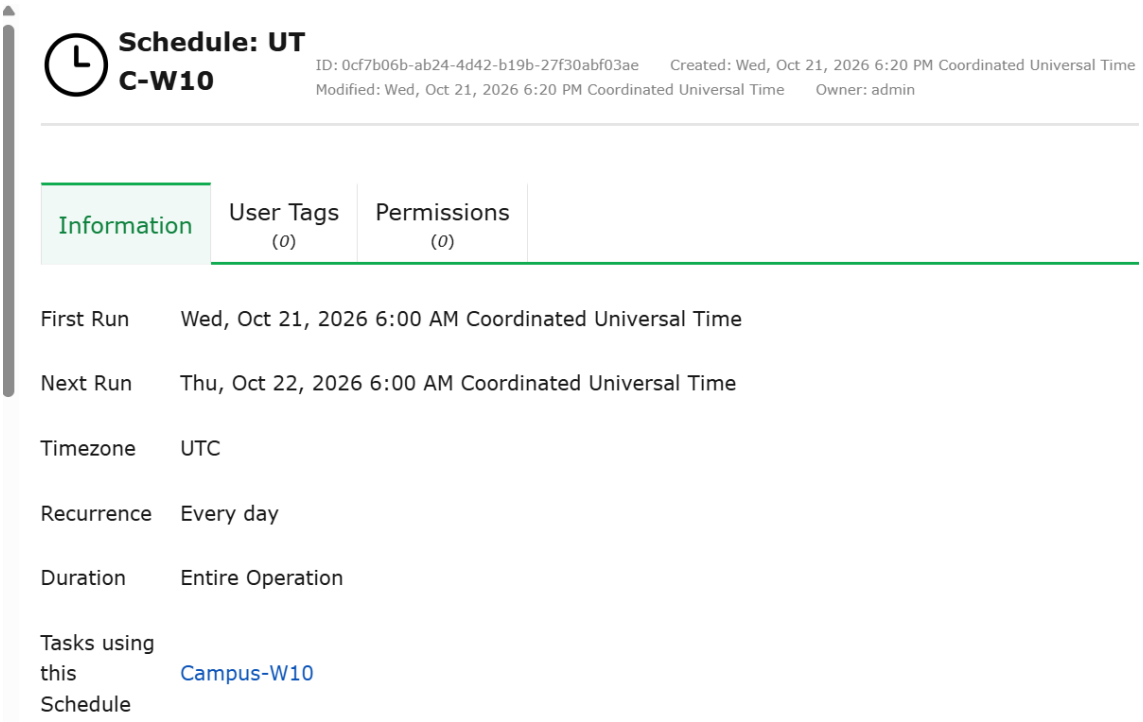


Figure 12 : Planification quotidienne « Campus-W10 » dans la GSA

Deux règles d'exploitation accompagnent ces planifications. Premièrement, une planification ne scanne que des actifs allumés : un poste éteint n'est pas « reporté », il est absent du rapport ; en production, les postes de travail se scannent donc en heures ouvrées, ou après réveil planifié ; le serveur de nuage, lui, est disponible en permanence. Deuxièmement, seul l'audit complet hebdomadaire est importé dans DefectDojo (engagement « Suivi hebdomadaire ») : importer aussi les scans quotidiens inonderait la plateforme et transformerait les ouvertures et clôtures automatiques en bruit.

Section 2 : Le tableau de bord KPI

Le tableau de bord (tableau 7) rassemble neuf indicateurs couvrant les trois familles du cahier des charges : volume et sévérité, efficacité de la remédiation, automatisation. Chaque valeur est datée

et rattachée à sa source : un KPI sans source datée n'est pas un KPI.

Tableau 7 : Le tableau de bord KPI et ses valeurs

Indicateur	Formule / source	Valeur relevée	Cible / seuil
1. Couverture	actifs scannés / inventaire	6 / 6, soit 100 %	100 % des actifs vitaux
2. Critiques campus (VLAN 30/31)	rapports GCE + matrice RBVM	MS17-010, SMB RCE, BlueKeep : corrigés et clôturés le 22/10	0
3. Critiques cloud (10.9.1.10)	rapport Full authentifié	0, confirmé par le Full du 21/10 (0 C / 0 H / 8 M / 3 L)	0
4. MTTR	(clôture − 1re détection), DefectDojo	14 jours (sudo : détecté le 02/10, fermé le 16/10)	Critique ≤ 30 j ; High ≤ 60 j
5. Tendance critiques/scan	cycles L1 → L2 → APRÈS → suivi	3C/1H → 9C/3H → 0C/0H → 0C/1H → 0C/0H	décroissante
6. Âge moyen des constats ouverts	1re détection – aujourd'hui	15-16 jours (la déduplication conserve la première détection)	critique > 30 j = alerte
7. Taux de remédiation automatisée	correctifs par playbook / total	Caméra : 100 % Ansible ; sudo : manuel	≥ 50 % à terme
8. Taux de remédiation	fermés / ouverts actifs	≈ 1,05 (20 fermés / ≈ 19 actifs, vague du 22/10)	> 1, croissant
9. Taux de régression	constats « Re-opened » / fermés	0 % (aucun constat corrigé n'est revenu)	≈ 0 %

Section 3 : Le tableur et le tableau de bord

La restitution visuelle combine un tableur (feuille « KPI » pour les neuf indicateurs, feuille « Historique des relevés » avec une ligne par scan et un graphique de tendance) et le tableau de bord intégré de DefectDojo pour le MTTR et la répartition par sévérité. La lecture de ces indicateurs alimente les décisions : la courbe de tendance raconte l'histoire complète du projet (la découverte voit la surface, l'authentifié révèle la profondeur, la remédiation referme) ; l'âge moyen, tiré vers le haut par les constats volontairement gelés, impose de formaliser les risques acceptés pour qu'ils sortent du pilotage courant ; le widget « ouvertes par semaine » du tableau de bord s'est révélé non fiable dans cette version et a été écarté au profit des listes filtrées, une règle d'or retenue : un indicateur se vérifie toujours contre la liste qu'il prétend résumer. La figure 13 montre le tableur, la figure 14 le tableau de bord DefectDojo.

	A	B	C	D	E
1	VMS — TABLEAU DE BORD KPI (Livrable 5, Obj. 6) — relevé du 21/10/2026				
2					
3	Indicateur	Formule / source	Valeur relevée	Cible / seuil	
4	1. Couverture	actifs scannés / inventaire P0	6 / 6 — 100 %	100 % des actifs vitaux	
5	2. Critiques VLAN 30 (campus)	rapports GCE + matrice RBVM (L3)	MS17-010, SMB RCE, BlueKeep : corrigés ET clôturés le 22/10 (re-scans « Clôturer les anciennes » — Criticals barrés, SLA rouges disparus)	0	
6	3. Critiques cloud (10.9.1.10)	rapport Full authentifié	0 — confirmé par le Full du 21/10 (0/0/8/3)	0	
7	4. MTTR / MTTM	(date de clôture − 1re détection) — DefectDojo	14 j (sudo : détecté 02/10, fermé 16/10)	Critique ≤ 30 j · High ≤ 60 j	
8	5. Tendance critiques/scan	cycles L1 → L2 → APRÈS → suivi	3C/1H → 9C/3H → 0C/0H → 0C/1H → 0C/0H (clôtures du 22/10)	décroissante	
9	6. Âge moyen des constats ouverts	DefectDojo (1re détection − aujourd'hui)	15-16 j (moyenne ≈ 15) — dédup conserve la 1re détection	critique > 30 j = alerte	
10	7. Taux de remédiation automatisée	correctifs par playbook / total	Camera : 100 % Ansible · sudo : manuel	≥ 50 % à terme	
11	8. Taux de remédiation (bilan L6)	constats fermés / ouverts actifs	≈ 1,05 — 20 fermés / ≈19 actifs (vague de clôture 22/10)	> 1, croissant	
12	9. Taux de régression	constats « Re-opened » / fermés	0 % — aucun Re-opened (non-régression prouvée)	≈ 0 %	
13					
14					

Figure 13 : Tableur KPI (indicateurs, cibles et historique des relevés)

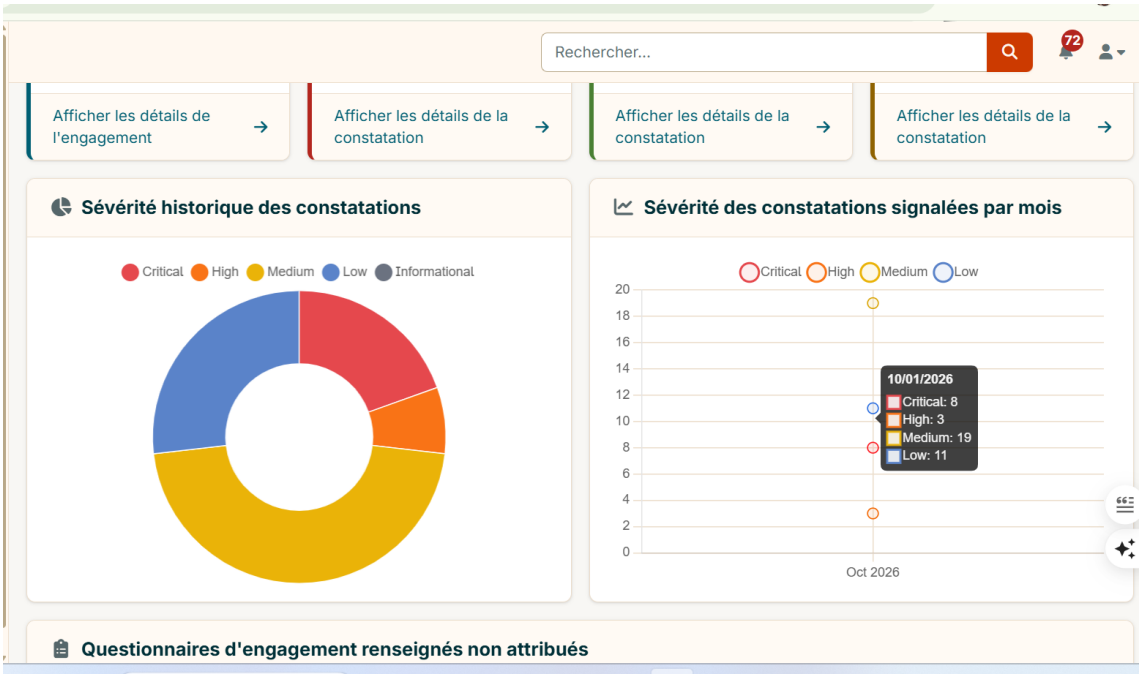


Figure 14 : Tableau de bord DefectDojo (répartition des sévérités)

Chapitre 6 : Évaluation de l’efficacité et continuité de service (Objectif 7)

Section 1 : La campagne de non-régression

Une correction n’a de valeur que si elle tient. La campagne de non-régression rejoue donc les scans sur toutes les remédiations, dans les mêmes conditions que leur détection (mêmes profils authentifiés, identifiants à jour). Les résultats confirment la stabilité de chaque actif : plus de MS17-010, d’exécution SMB à distance ni de BlueKeep sur les postes Windows 7 corrigés (0 C / 0 H / 1 M / 2 L et 0 C / 0 H / 0 M / 2 L) ; serveur cloud toujours propre (0 C / 0 H / 8 M / 3 L) ; postes Windows 10 stables (0 C / 0 H / 2 M / 1 L chacun) ; caméra stable et assumée (0 C / 1 H / 8 M / 4 L). Les contrôles externes complètent le tableau : le contrôle nmap des services d’administration confirme le telnet fermé et SSH maîtrisé (figure 15), et les clôtures tardives des constats critiques Windows (réimports « Clôturer les anciennes ») ont fait disparaître les dépassements de SLA signalés en rouge dans le tableau de constats ; la figure 16 montre les clôtures en chaîne obtenues après le réimport de validation.

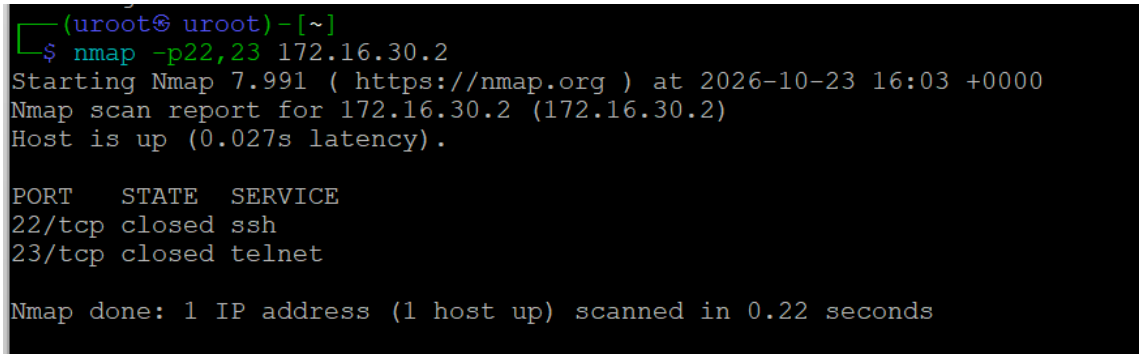


Figure 15 : Contrôle nmap des services d’administration (telnet fermé)

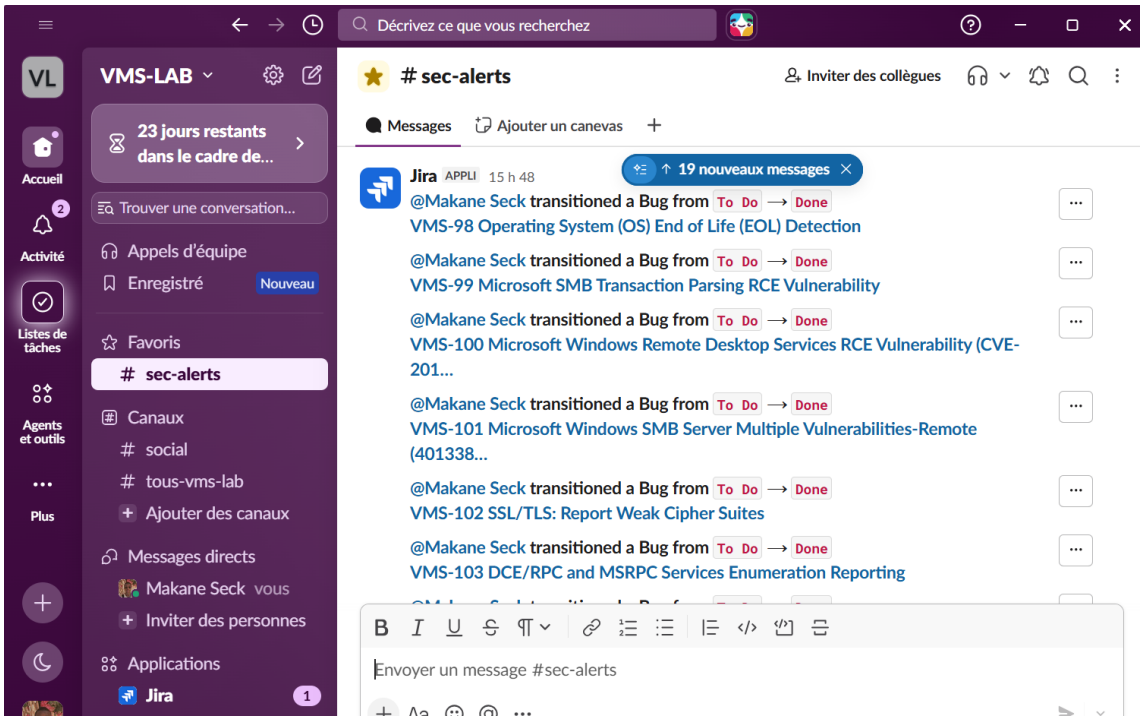


Figure 16 : Constats Windows clôturés en chaîne après réimport avec « Clôturer les anciennes »

Section 2 : Le piège telnet (la boucle complète en direct)

Pour prouver que la boucle détecte aussi le retour du mal, un piège volontaire a été tendu : le telnet d'administration, fermé depuis la première remédiation, a été replanté sur le commutateur SW-MET, comme un pare-feu mal refermé après une intervention. La séquence complète a alors été exécutée : scan ciblé, détection du constat telnet, création du ticket et alerte dans le canal d'équipe, remédiation (rétablissement de « transport input ssh »), re-scan et clôture automatique du ticket. Cette démonstration, réalisée sur un actif dont l'engagement était volontairement vierge, valide l'intégralité de la chaîne de la détection à la preuve de correction. La figure 17 retrace la séquence complète : détection, ticket VMS-120, clôture automatique.

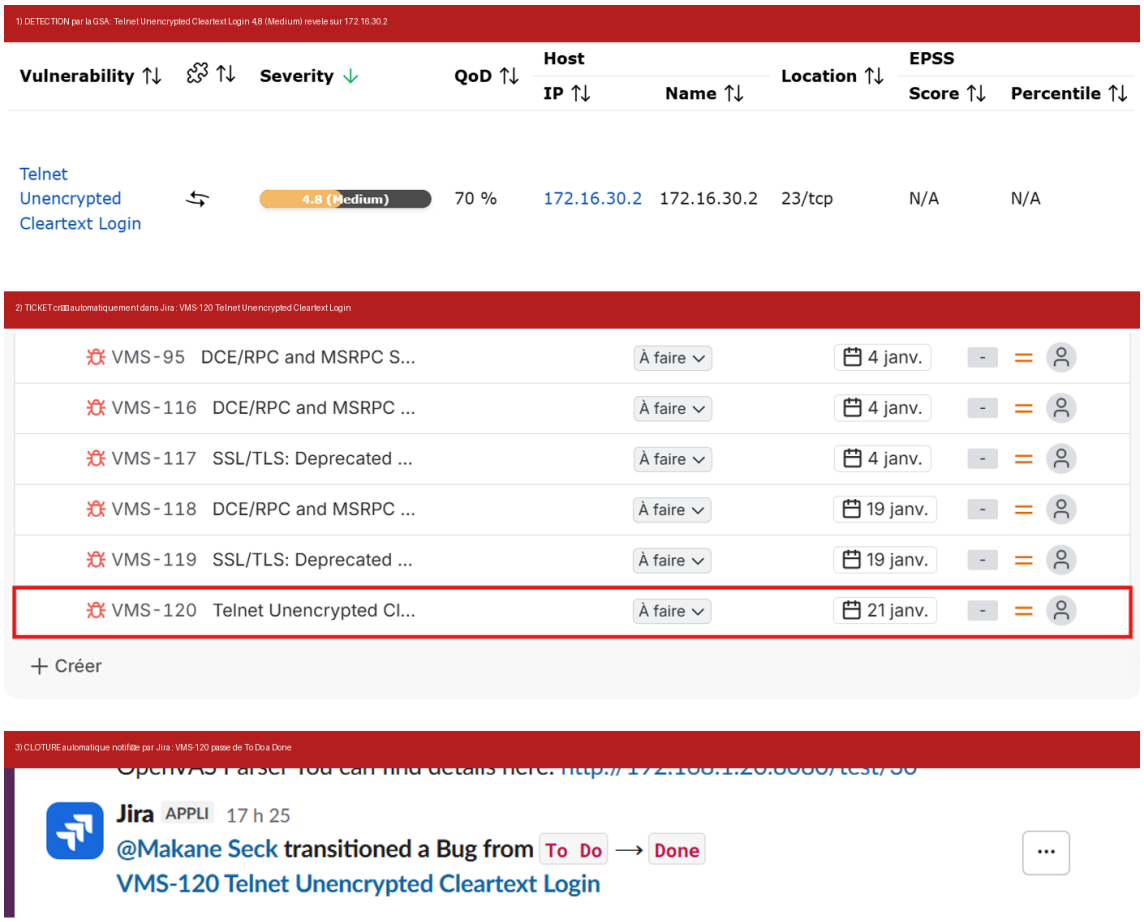


Figure 17 : Le piège telnet (détection, ticket VMS-120 et clôture automatique)

Section 3 : La continuité de service (bascule de la double liaison)

La surveillance ne sert pas si la plateforme meurt avec la panne : la disponibilité fait partie de l'efficacité. La liaison principale (MADIYANA) a donc été coupée au port du cœur, et la table de routage a basculé vers la liaison de secours (NDAM-MBACKÉ) via la route flottante, avec seulement quelques paquets perdus le temps du basculement. Trois constats valident la continuité : la connectivité Internet simulée revient d'elle-même ; le tunnel chiffré vers le nuage, porté par un segment indépendant, ne bouge pas ; et un scan de vulnérabilités lancé pendant la coupure se termine normalement. La restitution de la liaison principale ramène la table de routage à son état nominal. Une précaution d'implémentation mérite d'être consignée : dans le simulateur, la

coupure doit être faite sur le port du cœur ; couper l'interface du routeur opérateur ne propage pas la rupture et le basculement ne se déclenche pas. La figure 18 compare les tables de routage avant et après la bascule.

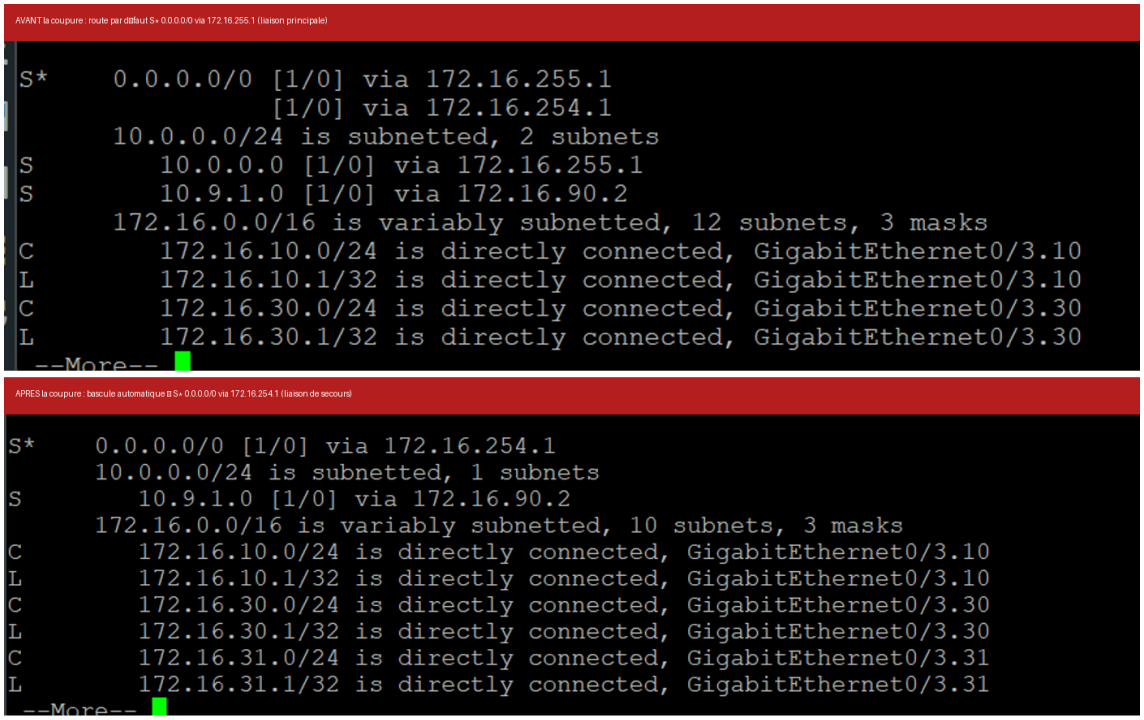


Figure 18 : Bascule de la double liaison (tables de routage avant et après coupure)

Section 4 : Le bilan d'efficacité

Le bilan final (tableau 8) reprend les indicateurs d'efficacité avec leurs valeurs mesurées en fin de projet : chaque cible est atteinte ou justifiée. Le taux de remédiation supérieur à 1 traduit une pile de constats qui décroît ; le zéro régression prouve que les corrections tiennent ; le MTTR de référence (14 jours pour le cas critique sudo) respecte le seuil de trente jours ; la couverture de 100 % garantit que rien de vital n'échappe à la surveillance. Le tableau de bord final (figure 19) affiche vingt constats clôturés au cours des sept derniers jours.

Tableau 8 : Le bilan d'efficacité final

Indicateur d'efficacité	Valeur finale	Cible	Verdict
Taux de remédiation	≈ 1,05 (20 fermés / ≈ 19 actifs)	> 1	Atteint
Taux de régression	0 % (aucun « Re-opened »)	≈ 0 %	Atteint
MTTR constaté	14 jours (sudo, détection → clôture)	Critique ≤ 30 j	Atteint
Couverture	6 / 6 actifs vitaux, soit 100 %	100 %	Atteint
Remédiation automatisée	Caméra 100 % par playbook ; sudo manuel	≥ 50 % à terme	En progression
Continuité de service	Bascule FO sans interruption des scans	aucune perte de service	Atteint

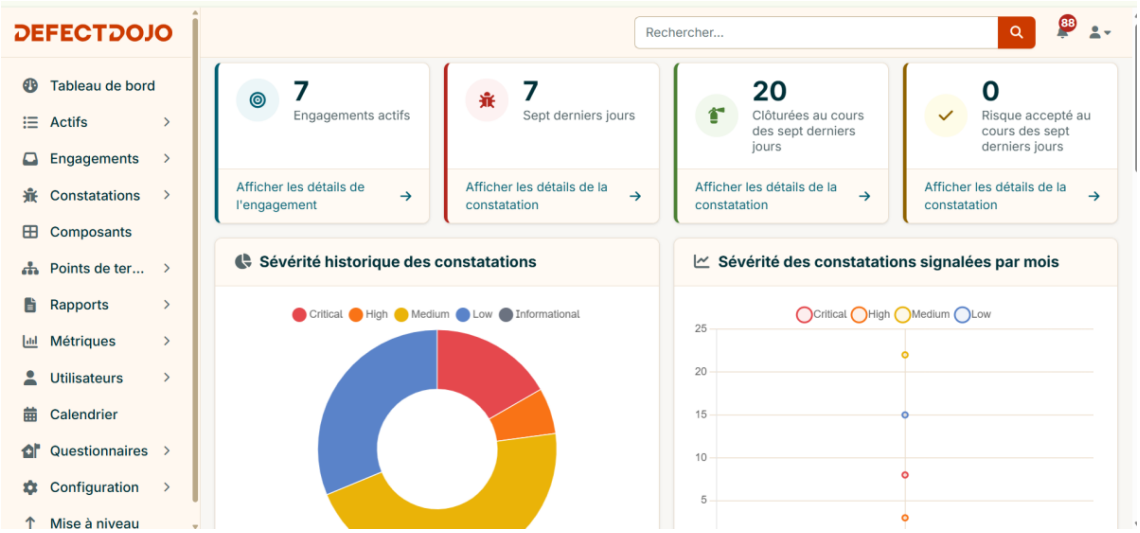


Figure 19 : Tableau de bord final DefectDojo (20 constats clôturés en sept jours)

Section 5 : Difficultés rencontrées et solutions

La richesse d'un projet se mesure aussi aux obstacles qu'il traverse. Le tableau 9 consigne les difficultés majeures et les solutions retenues, chacune ayant été vécue, diagnostiquée et validée au laboratoire.

Tableau 9 : Principales difficultés rencontrées et solutions retenues

Difficulté rencontrée	Solution retenue
Erreurs 401 en rafale vers l'API Jira	Utiliser un jeton classique d'utilisateur (portail de compte Atlassian), pas une clé d'administration d'organisation ; validation préalable par appel d'identité
Erreur « Misconfigured default issue type » à la liaison Jira	Les projets « gérés par l'équipe » ne sont pas couverts par les métadonnées : recréer le projet « géré par l'entreprise » (clé VMS)
Tickets non créés malgré les imports (silence)	« Push All Issues » ne pousse que les constats Actifs ET Vérifiés : réglage d'import « Verified = Force to True »
77 tickets créés d'un coup au lieu de 13	Exporter le CSV depuis la page du rapport (filtres hérités) et non depuis la vue globale ; purge puis règle de cohérence des décomptes à chaque maillon
Notifications Slack silencieuses malgré webhook et variable d'environnement	Inspection du code : API Slack codée en dur (jeton de bot et canal en base) → application Slack native « defectdojo », jeton xoxb, portée « système » de la matrice de notifications
pip3 refusé par la distribution (PEP 668)	Installer Ansible et passlib par les paquets de la distribution ou dans un environnement virtuel
Modules Ansible inutilisables sur la caméra (Python embarqué trop ancien)	Tâches « raw » exclusives (aucun Python distant), élévation par sudo, idempotence par fichiers marqueurs
Playbook non idempotent (re-modification à chaque passe)	Chaque tâche teste son marqueur avant d'agir ; hash de mot de passe à sel fixe
Perte d'Internet après configuration de la seconde interface du scanner	Une seule route par défaut (côté réseau local) ; les réseaux du lab passent par des routes spécifiques
Widget de tableau de bord non fiable (axe et zéros incohérents)	Vérifier tout indicateur contre la liste filtrée qu'il résume ; écarter le widget

Section 6 : Limites assumées et perspectives

Trois résiduels restent ouverts, par choix documenté : les huit constats moyens de noyau du serveur cloud, conséquence du scénario « OS gelé » (le risque est accepté et formalisé dans la plateforme) ; le constat haut sudo de la caméra, accepté au regard de son exposition limitée (VLAN d'accès uniquement, SSH réservé à l'audit, page durcie) ; sa mise à jour de firmware ou un second playbook constituant la perspective ; les protocoles TLS anciens des postes Windows, priorisés bas. Au-delà, les perspectives de production sont claires : déporter la remédiation des serveurs vers Ansible/AWX avec déclenchement par webhook dès l'apparition d'une CVE donnée ; compléter les scans par des agents locaux pour les ordinateurs portables nomades ; intégrer la veille EPSS à la priorisation ; et étendre la couverture aux autres bâtiments de l'université selon la même trame.

Conclusion générale et perspectives

Ce projet a conduit, de bout en bout, à la conception et à la mise en place d'un système de gestion des vulnérabilités pour l'université : un laboratoire EVE-NG de quinze nœuds reproduisant une architecture de centre de données à double liaison opérateur ; un scanner Greenbone déployé au sein du réseau ; des campagnes de découverte puis d'audit authentifié révélant l'exposition réelle du parc ; une centralisation DefectDojo avec déduplication et suivi des délais ; une priorisation fondée sur le risque croisant CVSS et criticité métier ; un workflow automatisé créant les tickets Jira, alertant le canal Slack, corrigeant par playbook Ansible et clôturant automatiquement à la preuve du correctif ; un tableau de bord de neuf indicateurs alimenté par des scans planifiés ; enfin, une campagne de non-régression et un test de bascule de liaison qui prouvent que la solution tient dans le temps et en panne.

Les résultats mesurés répondent point par point aux objectifs : couverture de 100 % des actifs vitaux ; révélation par l'authentifié de presque le triple des constats critiques ; tendance des critiques décroissante jusqu'à zéro ; MTTR de 14 jours sur le cas critique de référence ; taux de remédiation d'environ 1,05 ; zéro régression ; bascule de la liaison principale sans interruption des opérations de surveillance. Au-delà des chiffres, la démarche a produit une méthodologie réutilisable : grilles de criticité, protocoles d'import, règles de planification, registre des difficultés : autant d'artefacts directement transposables au réseau de production de l'université.

Les limites assumées (système gelé, objet connecté accepté, TLS anciens) sont documentées et portent leurs perspectives : montée de niveau des systèmes, seconde vague d'automatisation avec AWX, agents locaux pour les postes nomades, intégration de la prédiction d'exploitation à la priorisation. En définitive, ce projet démontre que la gestion des vulnérabilités n'est ni un outil ni un scan, mais un processus permanent (détecter, prioriser, corriger, prouver, recommencer) que l'université dispose désormais à la fois des fondations techniques et du pilotage par les indicateurs pour l'inscrire dans la durée.

Références bibliographiques

• FIRST. (2019). Common Vulnerability Scoring System version 3.1: Specification Document. Forum of Incident Response and Security Teams.

• FIRST. (2021-). Exploit Prediction Scoring System (EPSS). Forum of Incident Response and Security Teams.

• NIST. (2012). SP 800-40 Rev. 3: Managing Enterprise Information Technology Acquisitions and Patching. National Institute of Standards and Technology.

• NIST. (2018). SP 800-60 Vol. 1 Rev. 2: Mapping Types of Information and Information Systems to Security Categories.

• ISO/IEC. (2022). ISO/IEC 27001:2022, Information security, cybersecurity and privacy protection: Information security management systems, Requirements.

• MITRE Corporation. (1999-). Common Vulnerabilities and Exposures (CVE).

• NIST. (2007-). National Vulnerability Database (NVD).

• Greenbone Networks. (2024). Greenbone Vulnerability Management (GVM/OpenVAS), Documentation administrative.

• OWASP Foundation. (2023). OWASP Vulnerability Management Guide (v1.0).

• Atlassian. (2024). Jira Cloud REST API and administration documentation.

• Red Hat, Inc. (2024). Ansible Documentation, Playbooks et idempotence.

• DefectDojo Project. (2024). DefectDojo Documentation, Integrations (Jira, Slack) et API v2.

WEBOGRAPHIE

• https://www.greenbone.net : Greenbone Networks (GVM/OpenVAS)

• https://docs.defectdojo.com : Documentation DefectDojo

• https://www.first.org : FIRST (CVSS, EPSS)

• https://cve.mitre.org : Référentiel CVE (MITRE)

• https://nvd.nist.gov : National Vulnerability Database (NIST)

• https://www.atlassian.com : Documentation Jira Cloud

• https://docs.ansible.com : Documentation Ansible

• https://www.eve-ng.net : Documentation EVE-NG

• https://nmap.org : Documentation Nmap

Annexes

Annexe A : Actifs du laboratoire et accès d'audit

Les identifiants d'audit utilisés par les scans authentifiés (comptes SMB des postes, SSH du serveur et de la caméra, SNMP public) sont consignés dans le registre des comptes du projet, tenu à jour à chaque rotation de mot de passe. Ce registre, comme tous les secrets du projet, n'est pas reproduit dans ce mémoire pour des raisons évidentes de sécurité.

Annexe B : Commandes essentielles

Contrôle ciblé des services d'administration : nmap -p22,23 : l'état fermé du telnet et la maîtrise du SSH se vérifient en une seconde. Interrogation de la plateforme : curl authentifié sur l'API v2 de DefectDojo (/api/v2/product_types/ etc.). Validation d'un jeton Slack : appel direct à chat.postMessage avant toute configuration. Élévation et rotation de mot de passe sur cible embarquée : usermod -p via tâche raw Ansible.

Annexe C : Protocole du piège telnet (non-régression active)

1. État attendu : nmap -p23 fermé. 2. Replant volontaire : conf t → line vty 0 4 → transport input telnet. 3. Scan ciblé → import dans l'engagement de l'actif (format Greenbone, Lab, Verified = Force to True) → constat telnet au niveau Medium → ticket et alerte automatiques. 4. Remédiation : transport input ssh. 5. Re-scan → réimport avec « Clôturer les anciennes » → ticket clôturé automatiquement. La boucle complète est ainsi prouvée sur un actif vierge.

Annexe D : Extrait du playbook de remédiation de la caméra

Le playbook (variante raw) comporte trois tâches : rotation du mot de passe administrateur (usermod avec hash SHA-512 à sel fixe, garde par marqueur), remplacement de la page d'administration par la page durcie (garde par marqueur), et vérification d'état du service web (garde sur l'état réel). Chaque tâche déclare sa condition de modification (changed_when) à partir des marqueurs de sortie, ce qui garantit l'idempotence : première passe avec deux tâches modifiées, seconde passe entièrement stable.

Annexe E : Chronogramme du projet

Le déroulement sur six semaines a suivi le chronogramme joint au dossier : semaine 1, cycle de vie et comparaison des scanners ; semaine 2, déploiement du scanner et premier scan ; semaine 3, scoring et priorisation ; semaine 4, workflow de remédiation ; semaine 5, tableau de bord KPI ; semaine 6, évaluation de l'efficacité, continuité de service et rapport final.

Annexe F : Guide du tableur KPI

Le tableur comporte deux feuilles : « KPI » (les neuf indicateurs avec formule, valeur datée et cible) et « Historique des relevés » (une ligne par scan : date, cycle, cible, comptages par sévérité, avec graphique de tendance). Une ligne est ajoutée à chaque cycle de scan ; les indicateurs de la

feuille KPI viennent de l’historique.

Annexe G : Dépôt GitHub du projet

L’intégralité des artefacts du projet est publiée en accès libre sur GitHub : documentation du laboratoire, configurations des équipements, playbooks Ansible, générateurs des livrables, rapports de scan et captures de preuves. Le dépôt vise à permettre à toute équipe universitaire de reproduire la démarche ; les mots de passe et jetons y sont volontairement masqués, et les modalités de réutilisation sont décrites dans le fichier README du dépôt.

Adresse du dépôt : https://github.com/seckmaks/projet-gestion-vulnerabilites



Figure 20 : Code QR du dépôt GitHub du projet

Un scan du code QR ouvre la page d’accueil du dépôt : code source, livrables, preuves de démonstration et tableur de pilotage.

Table des matières

Dédicace 1
Remerciements 2
Résumé 3
Abstract 4
Abréviations et acronymes 5
Sommaire 7
Liste des figures 7
Liste des tableaux 7
Introduction générale 7
Chapitre 1 : Généralités sur la gestion des vulnérabilités 7
Section 1 : Définition de quelques concepts clés 11
Section 2 : Standards et référentiels 11
Section 3 : Les outils de la chaîne 11
Section 4 : Le cycle de vie de la vulnérabilité 12
Chapitre 2 : Environnement du projet 13
Section 1 : L'architecture réseau de référence 13
Section 2 : Le laboratoire EVE-NG 14
Section 3 : Préparation des cibles et vulnérabilités plantées 15
Chapitre 3 : Détection, centralisation et priorisation (Objectifs 1 à 4) 16
Section 1 : La découverte réseau (L1) 16
Section 2 : Les scans authentifiés (L2) 16
Section 3 : La centralisation dans DefectDojo 17
Section 4 : Le scoring RBVM 18
Chapitre 4 : Le workflow de remédiation automatisé (Objectif 5) 19
Section 1 : Architecture du workflow 19
Section 2 : Le ticketing automatique avec Jira 19
Section 3 : Les notifications Slack natives 20
Section 4 : La remédiation par Ansible 21
Section 5 : La boucle de clôture automatique 22
Chapitre 5 : Le pilotage par les indicateurs (Objectif 6) 23
Section 1 : Les scans planifiés 23
Section 2 : Le tableau de bord KPI 23
Section 3 : Le tableur et le tableau de bord 24
Chapitre 6 : Évaluation de l'efficacité et continuité de service (Objectif 7) 26
Section 1 : La campagne de non-régression 26
Section 2 : Le piège telnet (la boucle complète en direct) 27
Section 3 : La continuité de service (bascule de la double liaison) 27
Section 4 : Le bilan d'efficacité 28
Section 5 : Difficultés rencontrées et solutions 29
Section 6 : Limites assumées et perspectives 30
Conclusion générale et perspectives 31
Références bibliographiques 32
WEBOGRAPHIE 32

Annexes 33
Annexe A : Actifs du laboratoire et accès d’audit 33
Annexe B : Commandes essentielles 33
Annexe C : Protocole du piège telnet (non-régression active) 33
Annexe D : Extrait du playbook de remédiation de la caméra 33
Annexe E : Chronogramme du projet 33
Annexe F : Guide du tableur KPI 33
Annexe G : Dépôt GitHub du projet 34
Table des matières 35